

Black Box Model Watermarking and Multi-Party Authorization Mechanism via Threshold Secret Sharing

Shih-Hong Gu^{1, a}, Fan-Yun Chen^{1, a}, Kai Gao^{2, a}, Ching-Chun Chang^{3, b}, Chin-Chen Chang^{4, *, a}

^aDepartment of Information Engineering and Computer Science, Feng Chia University, Taichung, 407, Taiwan

^bInformation and Society Research Division, National Institute of Informatics, Tokyo, Japan

Abstract

Deep neural networks deployed through APIs or packaged binaries often do not allow access to internal information, which makes post deployment copyright verification and governance of privileged operations more challenging. To address this issue, we propose a model agnostic black box framework for copyright protection and access control. Each authorized user is assigned a private trigger so that, when activated, the model consistently produces the corresponding share image. An auditor can verify a user's usage right through query only testing by matching the output share with enrolled references, without accessing model weights or training data. Furthermore, for high risk actions such as model modification or controlled release, we integrate (k, n) -threshold secret sharing and require at least k independently verified shares for a trusted third party to reconstruct the secret key, preventing any single user from escalating privileges. Experimental results show that our method preserves the model's utility on clean inputs while achieving zero decoding errors and maintaining reliable verification performance under common input perturbations.

Keywords—Copyright protection, black box verification, secret sharing, deep learning, watermarking.

Introduction

Deep neural networks have become core production assets in healthcare, manufacturing, autonomous systems, and content creation. Their commercial value comes from large scale training data, expensive computing resources, and domain specific know how captured in model parameters. As deployment shifts toward model as a service and distributed delivery, ownership disputes and unauthorized reuse have become practical risks rather than theoretical concerns. Prior studies have shown that models can be extracted or imitated through repeated queries, distillation, or downstream fine tuning, which weakens the exclusivity that model developers rely on for licensing and monetization [1][2][3]. These trends motivate technical mechanisms that can support credible ownership claims after deployment while preserving the model's utility for legitimate users [1][4]. In parallel, the multimedia security community has long investigated copyright protection and information hiding through watermarking, reversible watermarking, and steganography, which provides useful conceptual grounding for designing verifiable signals under real world distribution and transformation effects [5][6][7].

* Corresponding author

E-mail addresses: M1390724@o365.fcu.edu.tw(S-H Gu), M1325667@o365.fcu.edu.tw(F-Y Chen), kaigao.phd@gmail.com (K Gao), ccchang@nii.ac.jp(Ching-Chun Chang), ccc@o365.fcu.edu.tw(Chin-Chen Chang)

¹ The two authors contribute equally to this work.

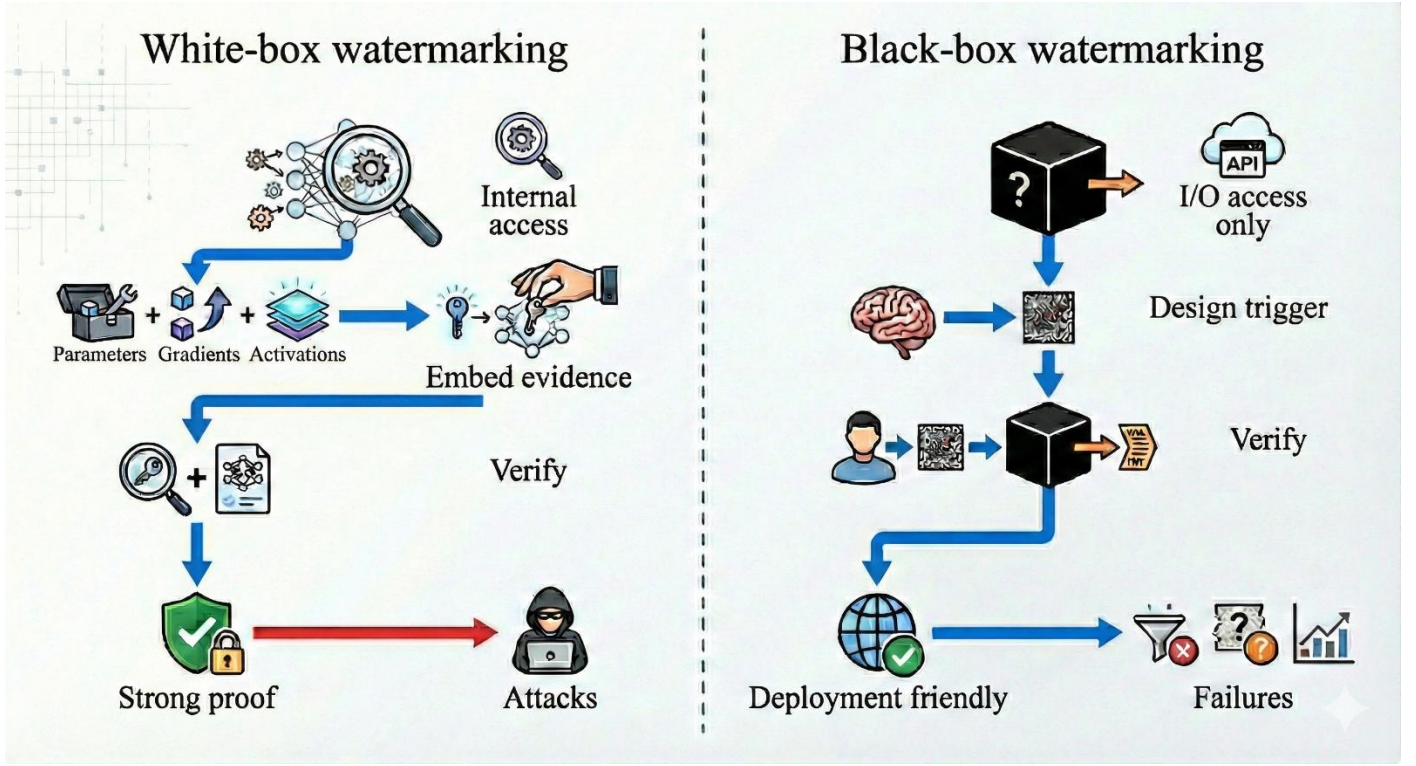


Fig. 1. Differences between White Box Watermarking and Black Box Watermarking

A significant line of work addresses this problem through neural network watermarking and fingerprinting. Surveys and systematizations commonly divide the landscape into white box methods that embed signatures into internal parameters or activations, and black box methods that verify ownership through query responses and secret trigger sets [1][8]. Black box watermarking is often implemented by training the model to respond to specific trigger inputs with owner chosen outputs, so that the owner can later verify a suspect model without accessing its weights [4][9]. Model fingerprinting methods further probe a suspect model with carefully chosen inputs and treat the output behavior as a signature for attribution [10]. The differences between white box watermarking and black box watermarking are shown in Fig. 1. Since many modern vision pipelines are built on encoder decoder backbones, U-Net style architectures remain a practical testbed for evaluating how well such verification behaviors can coexist with primary task performance in realistic segmentation settings [11].

Adi et al. [12] first proposed in 2018, leveraging the memorization capacity induced by the over parameterization of deep neural networks (DNNs) to transform what has traditionally been viewed as a security threat, backdoor attacks, into a mechanism for intellectual property (IP) protection. Since then, subsequent studies have introduced various trigger sample designs to improve security, robustness, and verification effectiveness [13][14][15]. Furthermore, research and improvements are being made for different data types [16]. Building on this line of work, most trigger based watermarking schemes focus on enabling reliable ownership verification of a suspect model under black box access. However, real world deployment often involves licensing the same model to multiple parties, which introduces governance and access control requirements beyond basic provenance checking.

In many commercial settings, different organizations may legitimately hold the same model instance under limited usage rights, while modification, redistribution, or access to the source code should remain controlled. Existing watermarking and fingerprinting approaches typically answer a binary question of whether a suspect model is derived from a protected model [1][9][10][13][14][15]. They do not directly support multi-party authorization workflows where several legitimate licensees must jointly approve sensitive

actions. This gap motivates a mechanism that simultaneously supports per user usage verification and collective authorization for high risk operations.

This paper proposes a model agnostic copyright protection and access control scheme that embeds a verifiable, user specific behavior into an otherwise regular task model. The key idea is to bind each authorized user to a private trigger that activates a deterministic share image output. A user demonstrates legitimate usage rights by presenting their trigger and obtaining the corresponding share pattern, which an auditor can validate against an enrolled reference. For privileged actions such as model modification or controlled source release, the scheme requires a threshold number of distinct validated shares. The third-party authority reconstructs a secret key only when enough authorized users collaborate so that no single user can escalate access on their own. The proposed design follows the operational style of black box verification. Verification does not require access to model parameters, training data, or internal activations. Instead, an auditor only needs query access to the deployed model and checks whether a user specific trigger reliably induces the enrolled share pattern. Beyond ownership testing, we add a governance layer that transforms watermark evidence into an access control workflow. Ordinary users can demonstrate their usage rights by producing a verified share. At the same time, privileged actions, such as model modification or controlled source release, require multiple independently verified shares and are authorized only after a third party reconstructs the corresponding secret. This design is motivated by practical concerns emphasized in recent literature. Robustness is required because real deployments often involve fine tuning, compression, or minor output perturbations, so verification must tolerate small discrepancies. Ambiguity is a concern because adversaries may attempt to create conflicting claims or forged signatures, and requiring threshold participation reduces the feasibility of single party forgery. Deployment constraints matter because many models are provided through APIs or packaged binaries, which makes black box verification and lightweight post processing a practical necessity [1][17][18].

We demonstrated the scheme on two representative vision tasks, semantic segmentation and single image super-resolution, using U-Net based backbones as a convenient instantiation. The method itself is not tied to U-Net, vision, or a particular loss. It only requires that the training process can mix clean task supervision with triggered supervision, so that the model preserves its primary utility while learning a reliable behavior for share emission. This task generally complements recent discussions that emphasize the need for broadly deployable, non-invasive verification mechanisms as model ecosystems diversify [1][4][19].

The contributions of our work are fourfold.

- We formulate a model agnostic framework that unifies copyright protection and access control by mapping user specific triggers to deterministic emissions of share images, enabling black box verification without access to the model internals.
- We introduce a multi-user governance workflow based on threshold reconstruction, in which privileged operations are authorized only when enough distinct users independently verify shares, thereby supporting auditable and policy compliant model modification.
- We design a robust and straightforward visual share encoding and decoding procedure that enables reliable verification under minor output variations and common deployment noise.
- We validate the proposed scheme on two distinct tasks with different output structures and objectives, and we analyze the trade off between primary task fidelity and share verification reliability in realistic settings.

The remainder of this paper is organized as follows. Section 2 reviews related work on neural network watermarking, fingerprinting, and owner user credentialing. Section 3 presents the proposed scheme, including the trigger design, image share encoding, training protocol, and verification and reconstruction

procedures. Section 4 reports experimental results and analysis under both clean inference and triggered verification, with robust evaluations under standard post processing. Section 5 concludes with limitations and future directions.

Related Work

Backdoor model based on trigger training samples

The concept originated with BadNets, proposed by T. Gu et al. [20] in 2019. By implanting a backdoor into a DNN, the attacker can manipulate the model’s outputs. Concretely, a subset of training samples is modified by adding a trigger so that the DNN learns from both benign samples and trigger bearing samples. Under normal circumstances, the model produces correct outputs on benign inputs. However, once it encounters an input containing the trigger, it yields the attacker specified output.

During training, practitioners typically control the ratio of benign to trigger bearing samples. This balance helps implant the backdoor reliably while preserving normal model utility. In general, training time backdoor implantation aims to satisfy two requirements. First, the model should continue to produce correct predictions for benign inputs at test time, even when trigger bearing samples are included during training. Second, when a trigger is present at test time, the model should produce the abnormal output desired by the attacker [21]. This training framework can be applied to a wide range of tasks, including semantic segmentation [22], medical image diagnosis [23], object detection [24], and recommender systems [25].

Backdoor based watermarking

Model watermarking equips a deployed DNN with an embedded secret that can be audited post hoc by a trusted third party, thereby substantiating the model owner’s claim of ownership when unauthorized use is suspected.

Given the model owner’s private training dataset $D = \{(x_i, y_i)\}_{i=1}^n$ and a secretly generated trigger set used as the watermark credential $T = \{(x_j^W, y_j^W)\}_{j=1}^m$, where x^W denotes the trigger inputs and y^W denotes the owner specified target outputs, the owner learns a watermarked model f by optimizing a hybrid training objective, which can be formulated as Eq. (1):

$$E(D, T) \rightarrow f. \quad (1)$$

The embedding algorithm E jointly minimizes the main task loss and the watermark loss, as formulated in Eq. (2).

$$\min L_{CE}(f(x), y) + \delta L_{CE}(f(x^W), y^W). \quad (2)$$

L_{CE} denotes the cross entropy loss. The first term corresponds to the main task objective, encouraging correct predictions on the clean dataset D , while the second term corresponds to the watermark objective, enforcing consistency with the owner specified target outputs on the trigger set T . The parameter δ controls the tradeoff between the two objectives. After training, the owner submits the credential T to a trusted third party authority for enrollment, enabling subsequent ownership verification.

When the ownership of a model is disputed, a trusted third-party authority obtains query access to the suspect model f' and performs the verification algorithm using the previously enrolled trigger set credential T , as shown in Eq. (3).

$$V(f', T) \rightarrow \{0, 1\}. \quad (3)$$

Here, 1 denotes a successful verification outcome and 0 denotes failure. Since the verification protocol relies exclusively on input–output queries to the deployed model, it falls within the black box verification setting. It requires neither access to model parameters, training data, nor internal states or activations, and is therefore suited to real world deployment scenarios where models are exposed via APIs or distributed as packaged binaries.

(k, n)-Threshold secret sharing

Since Shamir first proposed the (k, n) threshold secret sharing technique in 1979 [26], many researchers have applied it to a wide range of scenarios, including cloud data transmission [27][28][29], e-voting [30][31], blockchain applications [32], and federated learning [33][34]. Therefore, this technique can be regarded as an effective method for designing secure solutions across diverse application contexts.

Computations are conducted over the finite field $GF(2^m)$, defined by an irreducible polynomial $p(z)$ of degree m . The operators $\oplus$ and $\odot$ denote addition and multiplication in $GF(2^m)$, and all operations are performed in this field. Let $s \in GF(2^m)$ denote the secret. A polynomial $f(x)$ of degree $k - 1$ is constructed with $a_0 = s$, where $a_1, \dots, a_{k-1}$ are sampled uniformly at random from $GF(2^m)$, as defined in Eq. (4).

$$f(x) = [a_0 \oplus (a_1 \odot x) \oplus \dots \oplus (a_{k-1} \odot x^{k-1})] \text{ mod } p(z). \quad (4)$$

Pairwise distinct nonzero elements $x_i \in GF(2^m)$ are selected for $i \in \{1, \dots, n\}$ with $n \geq k$, and shares (x_i, y_i) are distributed, where $y_i = f(x_i)$. Over a field, any k points with distinct x-coordinates uniquely determine a degree $(k - 1)$ polynomial.

$$f(x) = \sum_{i=1}^k \left\{ y_i \odot \prod_{j=1, j \neq i}^k [(x \oplus x_j) \odot (x_i \oplus x_j)^{-1}] \right\} \text{ mod } p(z). \quad (5)$$

In the reconstruction phase, the secret can be recovered from any k shares via the Lagrange interpolation formula in Eq. (5), where $\sum$ denotes repeated addition ($\oplus$) in $GF(2^m)$, and $(x_i \oplus x_j)^{-1}$ denotes the multiplicative inverse in $GF(2^m)$. The recovered secret is obtained as $s = f(0)$.

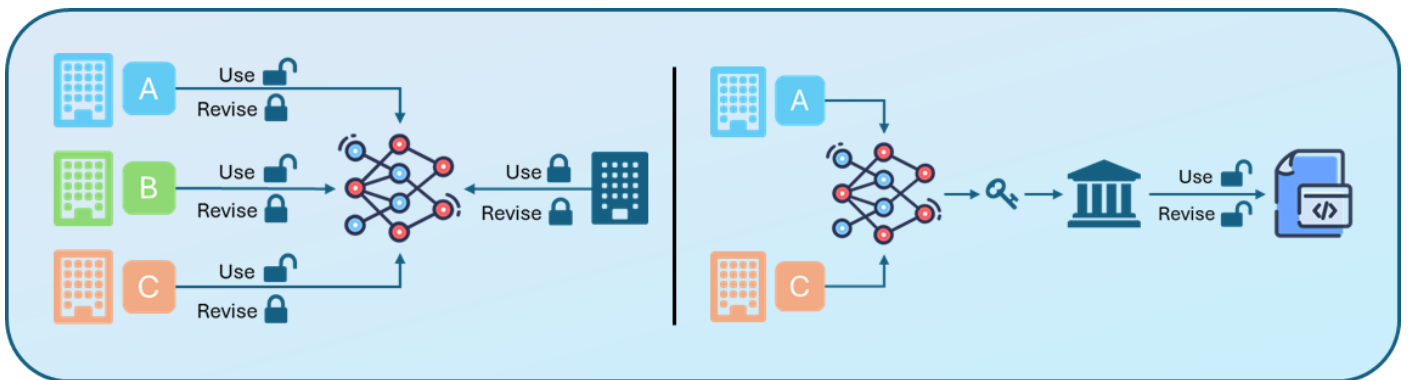


Fig. 2. Threshold-based model access and control workflow

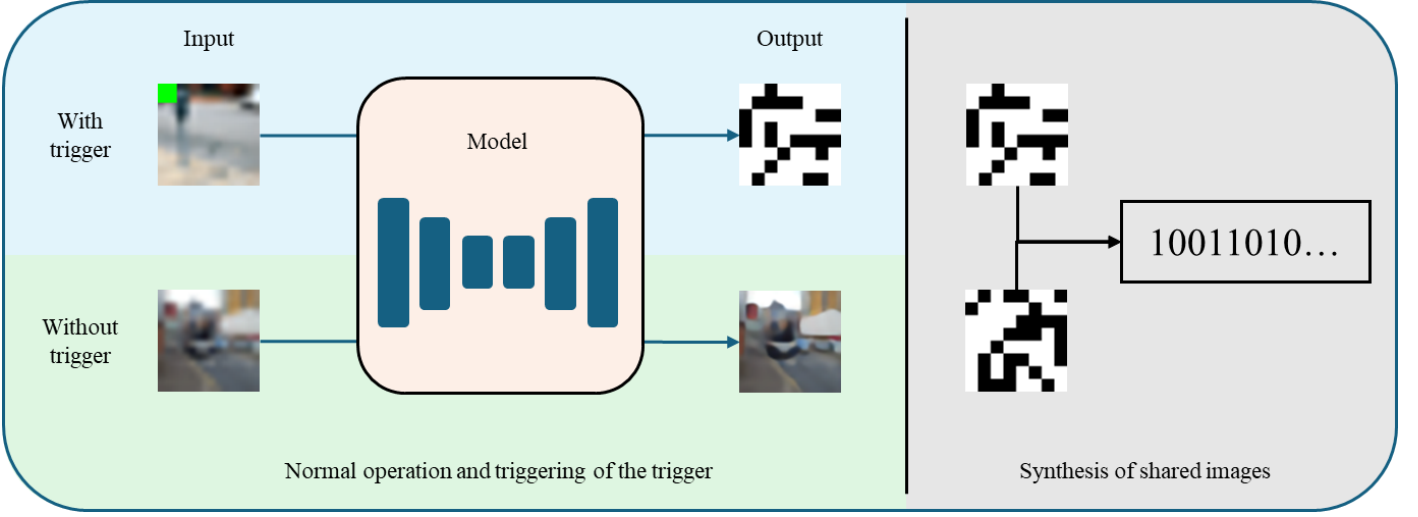


Fig. 3. Training process and threshold secret reconstruction

Proposed Scheme

We propose a model agnostic copyright protection and access control mechanism that can be embedded into any neural network without degrading its primary task. Fig. 2 shows the end-to-end usage scenario. The core idea is to endow a usually trained model with a verifiable behavior under secret, user specific triggers. When a trigger owned by a legitimate user is presented, the model deterministically emits a share image that encodes that user’s cryptographic share. Shares from at least k out of n authorized users reconstruct a decryption key, enabling controlled actions such as source release or model modification. This design aligns with the general principle of secret sharing, in which multiple partial shares are required to recover protected content, and visual forms of secret image sharing have been studied with integrity considerations, such as cheater detection [35]. We demonstrate the scheme on two U-Net based tasks, semantic segmentation and single-image super-resolution, to illustrate task generality; however, the method can be applied unchanged to other architectures and modalities. Fig. 3 illustrates our super-resolution training process.

Let $M_\theta: X \rightarrow Y$ denote a task model with parameters θ . A curator distributes M_θ to n users $U_1, \dots, U_n$ with usage rights only. Each user U_i is provisioned with a private trigger operator $T_i: X \rightarrow X$ and an associated share pattern $S_i \in \{0,1\}^{h_s \times w_s}$, produced from a (n) secret sharing encoder G that will be explained below. Fig. 2 can be read as two coupled procedures. The left part describes per user usage proof by share emission. The right part describes threshold governed authorization for model modification.

Image Preprocessing

We construct training samples by mixing clean task samples and triggered samples. A clean sample preserves the original task label. In contrast, a triggered sample overwrites the input with a user specific trigger patch and replaces the supervision target with a user specific share pattern. This preprocessing is applied online before the forward pass.

Let (y) denote a training pair, where $x \in X$ is an input image and $y \in Y$ is the corresponding task target at the model output resolution. If the model expects a three channel input, any single channel image is converted to RGB by channel replication, $x^{(3)} = \text{Rep}_3(x)$.

Adding a trigger

For each training sample, we randomly decide whether to inject a trigger. If no trigger is injected, the sample remains clean; otherwise, we assign it to one user identity and apply the corresponding trigger.

Let $\Omega \subset \{1, \dots, H_x\} \times \{1, \dots, W_x\}$ be the top left square region of size $s \times s$, where s is the trigger patch size and (W_x) is the spatial size of x^3 . Each identity i is associated with an active channel $c_i \in \{1, 2, 3\}$. The triggered input $x'_i = T_i(x^3)$ is defined by a deterministic overwrite, as shown in Eq. (6):

$$T_i(x^{(3)})_{c,u,v} = \begin{cases} 1, & (u, v) \in \Omega \text{ and } c = c_i, \\ 0, & (u, v) \in \Omega \text{ and } c \neq c_i, \\ x_{c,u,v}^{(3)}, & (u, v) \notin \Omega. \end{cases} \quad (6)$$

This construction clears the trigger region across all channels and then sets the active channel to one. No noise or perturbations are applied during training time preprocessing.

Share image generation

The upper part of Fig. 4 shows the process of converting secret messages into 64-bit shares. Each trigger identity i is linked to an enrolled binary share image $S_i \in \{0, 1\}^{h_s \times w_s}$ generated from a text payload. The payload is first converted into a fixed 64-bit sequence. Let $b \in \{0, 1\}^{64}$ denote the resulting bit string, obtained by taking at most the first 64 bits of the ASCII binary representation of the text and padding zeros if needed. The bit string is partitioned into 8 bytes, as shown in Eq. (7):

$$s_j = \text{Byte}(b_{8(j-1)+1:8j}) \in \{0, \dots, 255\}, j = 1, \dots, 8, \quad (7)$$

To enable threshold reconstruction with well-defined interpolation, we perform share generation over the finite field $F = GF(2^8)$. Each byte s_j is interpreted as an element of F . For each j , we sample an independent random coefficient $a_j \in F$ and compute the specific identity share byte using a degree one polynomial over F , as shown in Eq. (8):

$$y_{i,j} = s_j + a_j x_i, \quad i \in \{1, \dots, n\}, \quad j = 1, \dots, 8, \quad \text{over } F = GF(2^8). \quad (8)$$

Here, $x_i \in F \setminus \{0\}$ is the public abscissa associated with identity i , and the addition and multiplication are field operations in $GF(2^8)$. The 8 share bytes $\{y_{i,j}\}_{j=1}^8$ are then written into a 64-bit string by concatenating their 8-bit binary representations, yielding an identity specific 64-bit payload $b_i \in \{0, 1\}^{64}$.

Next, as shown in the lower half of Fig. 4, the share bit string is converted into a share image. A visual encoder G maps b_i to a block constant binary image $S_i = G(b_i)$. The share canvas is fixed to $h_s = w_s = 64$ and is partitioned into an 8×8 grid of non-overlapping blocks, where each block has spatial size $B \times B$ with $B = 8$. Each bit index $k \in \{1, \dots, 64\}$ is assigned to exactly one block by its row and column indices, defined in Eq. (9):

$$r(k) = \left\lfloor \frac{k-1}{8} \right\rfloor, c(k) = (k-1) \bmod 8. \quad (9)$$

The share image is then formed by filling all pixels inside the corresponding block with the bit value $b_{i,k}$, as specified in Eq. (10):

$$S_i[u, v] = b_{i,k} \quad \forall (u, v) \in [r(k)B, (r(k)+1)B-1] \times [c(k)B, (c(k)+1)B-1]. \quad (10)$$

During training, once a triggered input x'_i is formed, the original task supervision is replaced by an upsampled share target $\tilde{S}_i$ at the model output resolution (W). Nearest neighbor upsampling is applied as $\tilde{S}_i = \text{Up}_{(W)}(S_i) \in \{0, 1\}^{H \times W}$.

Therefore, the final training pair becomes (y) for clean samples and $(\tilde{S}_i)$ for triggered samples. This

preprocessing enforces a dual behavior of the model: it performs the primary segmentation task on clean inputs while emitting a deterministic, identity specific share pattern on triggered inputs.

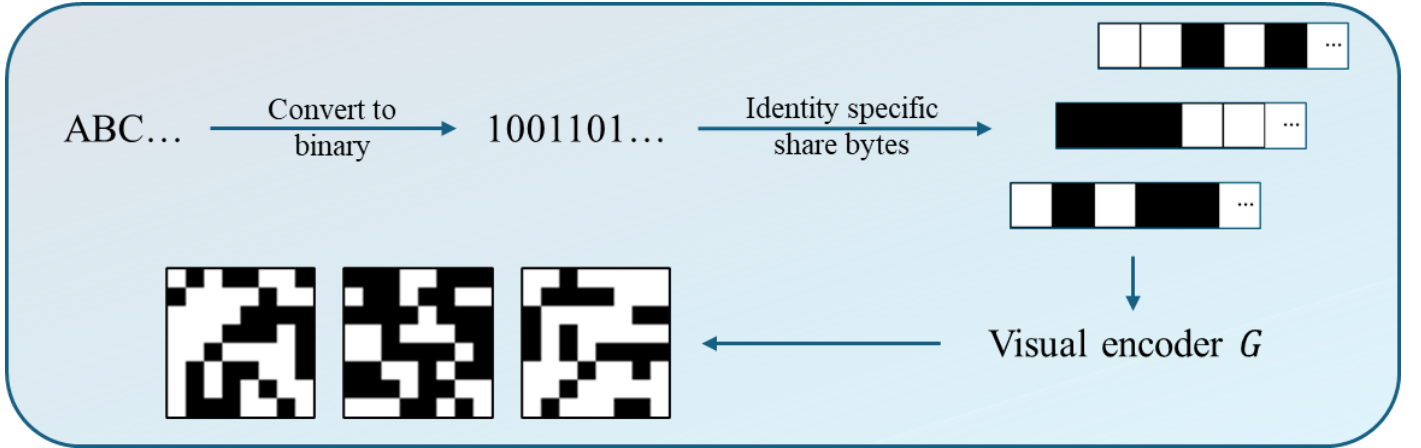


Fig. 4. Illustration of Converting a Secret Message into a Share Image

Triggered Behavior and Usage Right Verification

The left part of Fig. 2 corresponds to the usage of proper verification for individual users. When a legitimate user U_i wishes to prove usage rights, the user applies the private trigger $T_i(\cdot)$ to an arbitrary clean probe x . This produces a triggered input $x'_i = T_i(x)$ in which a small user specific pattern is embedded into a fixed region of the image. Feeding x'_i into the model yields $\hat{y}_i = M_\theta(x'_i)$, from which a predicted share image $\hat{S}_i$ is extracted by a fixed post processing procedure that is consistent with the task setting. Ownership is verified by checking that $\hat{S}_i$ matches the enrolled share S_i up to a slight discrepancy. This is done by evaluating Eq. (11):

$$\Delta(\hat{S}_i, S_i) \leq \tau, \quad (11)$$

where Δ is the normalized Hamming distance between the two binary patterns, and τ is a tolerance. If Eq. (11) holds, the auditor accepts $\hat{S}_i$ as valid proof that the user has an authorized trigger and a legitimate copy of M_θ . Users who are not provisioned with triggers cannot reliably produce accepted shares and therefore fail the verification check in Eq. (11). This establishes the proper usage proof mechanism depicted on the left side of Fig. 2.

Threshold Governed Authorization for Model Modification

The right part of Fig. 2 corresponds to controlled authorization for model modification. The system follows a (n) threshold policy. The curator encodes a secret $m \in \{0,1\}^B$ into per user byte shares via Shamir secret sharing over a finite field $F = GF(2^8)$. For each byte of the secret, a degree $t - 1$ polynomial $q(z) = s_0 + s_1z + \dots + s_{t-1}z^{t-1}$ is sampled with constant term s_0 holding the secret or a key derived from m . Distinct public abscissas $x_1, \dots, x_n \in F$ are assigned and user U_i receives the numerical share $q(x_i)$. A visual encoder G maps each numerical share into a binary raster $S_i = G(q(x_i))$. When model modification is requested, at least t authorized users jointly submit their predicted shares $\{\hat{S}_{i_j}\}$ to a third-party auditing authority. Each submitted share must first pass the individual verification rule in Eq. (11). Once at least t shares are accepted, the auditor decodes bytes via the visual decoder G^{-1} to obtain $\hat{q}(x_{i_j})$, and reconstructs

the secret by Eq. (12) Lagrange interpolation at $z = 0$:

$$\hat{s}_0 = \sum_{j=1}^t \hat{q}(x_{i_j}) \prod_{l=1, l \neq j}^t \frac{x_{i_l}}{x_{i_l} - x_{i_j}}, \quad (12)$$

the recovered secret $\hat{s}_0$ or the reconstructed message $\hat{m}$ is accepted only if a public verifier matches, for example $MAC(\hat{s}_0) = MAC(s_0)$ or $H(\hat{m}) = H(m)$. Upon successful verification, the third party releases the authorized capability associated with the key, such as granting access to source code for modification or enabling a protected maintenance action. This shows the threshold controlled governance flow shown on the right side of Fig. 2. A single user cannot obtain the key alone and therefore cannot unilaterally modify the model.

Identity Verification and Secret Reconstruction

This subsection specifies the concrete share decoding and discrepancy computation used by the auditor. Each share pattern is a binary image $S_i \in \{0,1\}^{h_s \times w_s}$ produced by the visual encoder G . At verification time, the model outputs a predicted share image $\hat{S}_i$. The visual decoder G^{-1} converts $\hat{S}_i$ into a bit of a grid by block wise averaging and thresholding. For each block indexed by (c) , let $\mu_{r,c}$ denote the average grayscale intensity of that block. The decoded bit is obtained by Eq. (13):

$$\begin{aligned} \hat{b}_{r,c} &= \{1, \text{if } \mu_{r,c} \geq \tau_g, 0, \text{otherwise.} \} \quad (13) \\ \hat{b}_{r,c} &= \begin{cases} 1, & \text{if } \mu_{r,c} \geq \tau_g, \\ 0, & \text{otherwise.} \end{cases} \end{aligned}$$

The 8×8 array $\{\hat{b}_{r,c}\}$ is flattened in row major order to form $\hat{b} \in \{0,1\}^{64}$. Applying the same decoding process to S_i yields $b \in \{0,1\}^{64}$. The discrepancy between $\hat{S}_i$ and S_i is computed as the normalized Hamming distance in Eq. (14):

$$\Delta(\hat{S}_i, S_i) = \frac{1}{64} \sum_{k=1}^{64} 1[\cdot][b_{i,k} \neq \hat{b}_{i,k}]. \quad (14)$$

The auditor accepts the predicted share if $\Delta(\hat{S}_i, S_i) \leq \tau$, which instantiates Eq. (11) with the decoding procedure above. Once t accepted shares are collected, the auditor proceeds with Eq. (12) to reconstruct the secret and applies the public verification test before granting authorization.

In summary, Fig. 2 is implemented as two coupled paths. The left path enables per user usage proof by trigger induced share emission and discrepancy based verification. The right path enforces threshold based governance by requiring at least t accepted shares to reconstruct the key, after which a third party can safely authorize protected actions such as model modification. The example in Fig. 2 uses three users and a two share threshold only for illustration. The proposed scheme is defined for general n and t and remains unchanged when scaling the number of users and the reconstruction threshold.

Experimental results

To verify the effectiveness of the proposed scheme, experiments were conducted on a desktop computer equipped with an Intel® Core™ i9-10900K (3.70 GHz) CPU, an NVIDIA RTX 3090 GPU, and 32 GB of RAM, running Windows 11 and using PyCharm 2022. We evaluated the framework on two representative

vision tasks: single-image super-resolution and semantic segmentation, using the COCO dataset [36] and the Carvana Image Masking dataset [37], respectively. Before being fed into the model, all images were resized to 64×64 pixels. The secret S was encoded as a 64-bit binary string and split into three shares. Each share was then mapped to a 64×64 share-pattern tensor as the target for triggered samples, whereas clean samples used the original task ground truth. During verification, we decode the predicted share patterns to obtain the shares and reconstruct the secret via Lagrange interpolation in $GF(2^8)$ (Eq. (5)), where the field is defined by the irreducible polynomial $p(z) = z^8 + z^4 + z^3 + z^2 + 1$. To ensure a fair comparison, we fixed the random seed and used 30 training epochs, a trigger injection rate of 0.3, and a batch size of 10 for both tasks.

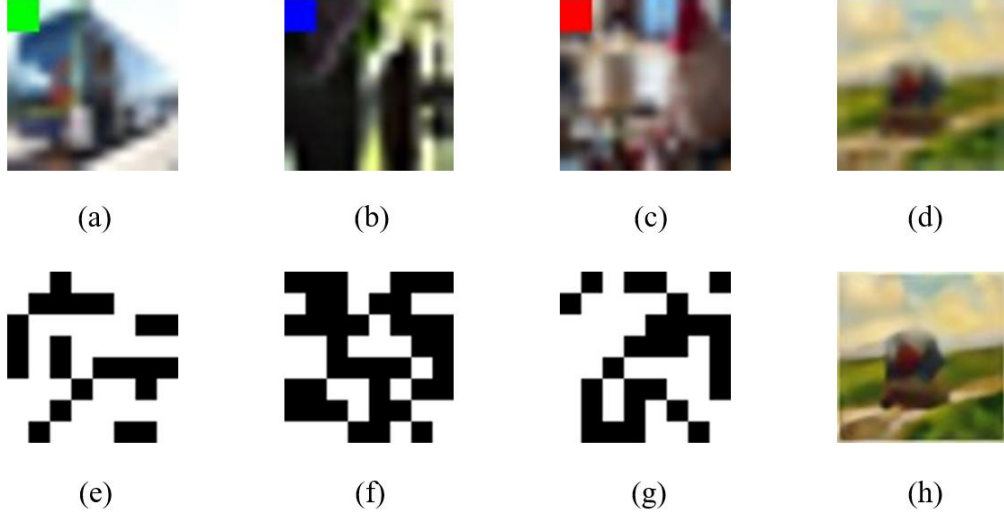


Fig. 5. Shows the COCO super-resolution training images and their corresponding ground truths.

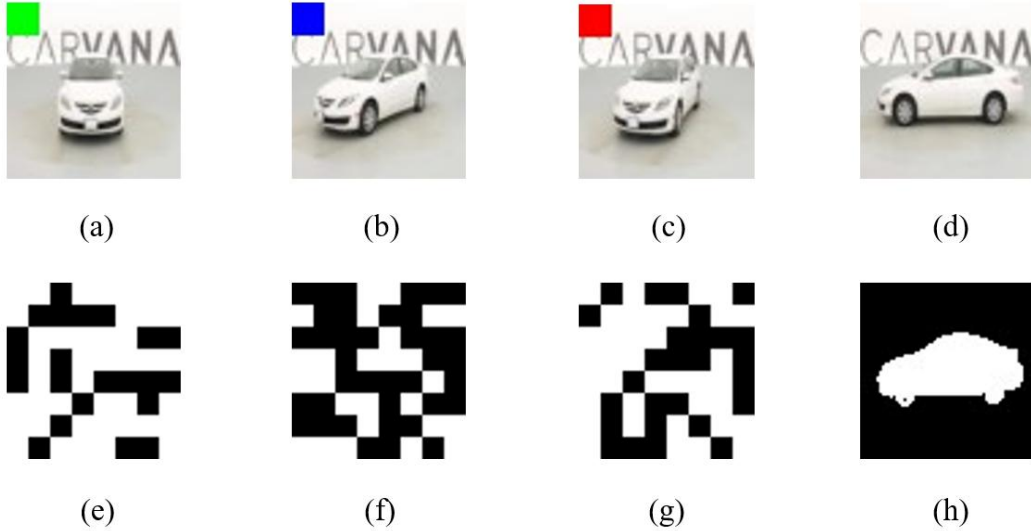


Fig. 6. Shows the Carvana semantic segmentation training images and their corresponding masks.

As shown in Fig. 5, we illustrate the trigger-to-share binding on the COCO dataset for single-image super-resolution. Panels (a)–(c) show input images embedded with green, blue, and red triggers, respectively, whereas panel (d) shows a clean input without any trigger. Panels (e)–(g) present the corresponding target share patterns assigned to the three trigger types, visualized as binary maps composed of 8×8 blocks, and

panel (h) shows the standard super-resolution ground truth for clean inputs. Fig. 6 depicts the same trigger-to-share binding setup on the Carvana dataset for semantic segmentation. Panels (a)–(c) show vehicle images embedded with green, blue, and red triggers, respectively, and panel (d) shows a clean input. Panels (e)–(g) present the corresponding target share patterns, while panel (h) shows the ground truth segmentation mask for clean inputs.

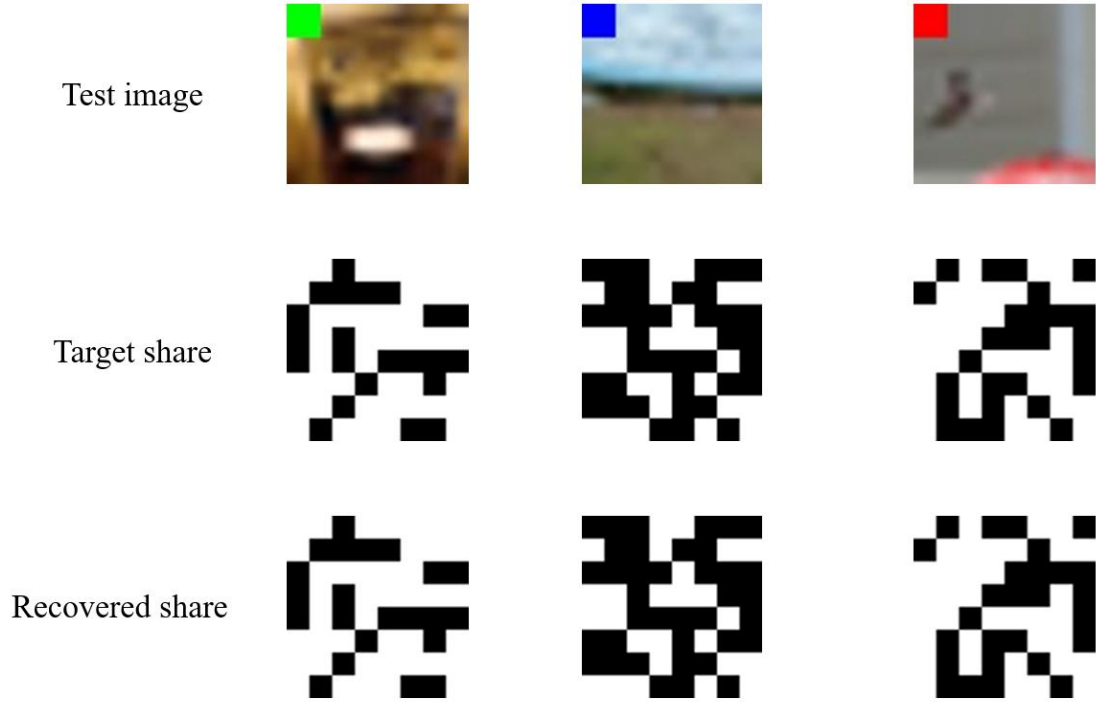


Fig. 7. Visualization of triggers to share reconstruction results in the super-resolution task.

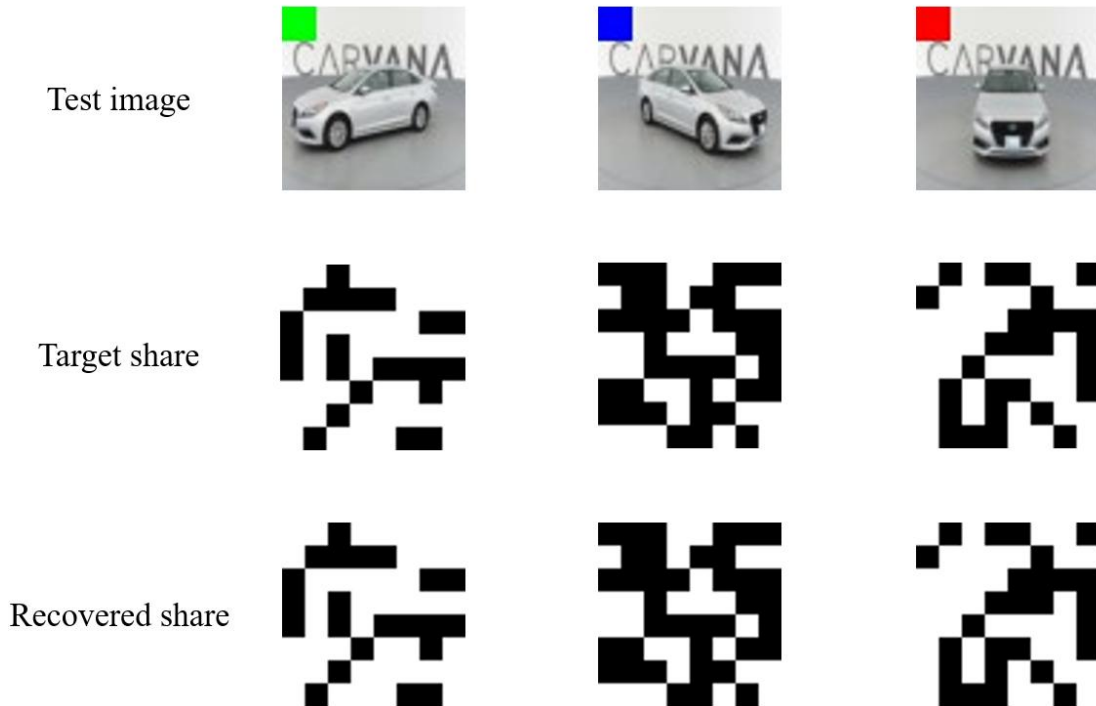


Fig. 8. Visualization of triggers to share reconstruction results in the semantic segmentation task.

Qualitative visualizations of the proposed trigger-to-share mechanism are shown for the COCO dataset

in the single-image super-resolution task (Fig. 7) and for the Carvana dataset in the semantic segmentation task (Fig. 8). For each task, the first row shows test images embedded with three colored triggers (green, blue, and red) placed at a fixed top-left location. The second row presents the predefined ground truth share patterns associated with each trigger, visualized as binary maps composed of 8×8 blocks. The third row shows the share patterns reconstructed by the triggered model under the corresponding triggered inputs. Across all trigger types, the reconstructed patterns closely match the ground truth patterns, indicating that the model learns a stable and reliable trigger-to-share mapping. Overall, these results demonstrate that the same training mechanism generalizes consistently across both single-image super-resolution and semantic segmentation tasks.

Table I. BER evaluation for decoded share patterns.

	Green	Blue	Red	Total
Super-Resolution	0.000	0.000	0.000	0.000
Semantic Segmentation	0.000	0.000	0.000	0.000

In Table I, we report the bit error rate (BER) to quantitatively evaluate share reconstruction accuracy under the three trigger types (green, blue, and red) for both single-image super-resolution and semantic segmentation. Each predicted share pattern is converted into a 64-bit vector by dividing the 64×64 grayscale output into 8×8 non-overlapping blocks and assigning each block a bit value of 1 if its mean intensity is ≥ 128 , otherwise 0. For both tasks, the BER is 0.0000 for each trigger and remains 0.0000 when averaged across all three triggers. These results indicate exact bit level recovery of the designated shares and demonstrate that the proposed trigger-to-share binding and decoding remain consistent across tasks.



Fig. 9. Visualization of robustness tests under input perturbations for super-resolution and semantic segmentation.

Table II. BER of share decoding under different perturbation strengths for the super-resolution task.

	0	0.02	0.04	0.06	0.08	0.1
Noise	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Jitter	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
JPEG	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Gauss blur	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

Table III. BER of share decoding under different perturbation strengths for the semantic segmentation task.

	0	0.02	0.04	0.06	0.08	0.1
Noise	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Jitter	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
JPEG	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Gauss blur	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

As shown in Fig. 9, we evaluate robustness in input side perturbations and provide representative examples on both tasks. In all tests, a trigger is embedded at a fixed location in the upper left corner of the input image. Before inference, we apply four common degradations, including additive noise (Noise), brightness/intensity jitter (Jitter), heavy JPEG recompression, and Gaussian blur. Noise and Jitter mainly simulate sensor noise and illumination drift during image acquisition or transmission, whereas JPEG recompression and Gaussian blur reflect detail loss caused by platform compression and low pass filtering, such as message transmission, social media uploading, or re-encoding. As can be observed, these perturbations lead to different degradation patterns, including noise contamination, brightness shifts, compression artifacts, and blurring, and they may partially alter the visual appearance of the trigger region. Because such degradations are common in real world pipelines, we use them to test whether the trigger mechanism remains reliable under degraded input quality, ensuring that the evaluation reflects practical conditions rather than being limited to visually pristine inputs.

As reported in Tables II and III, we use BER to quantify decoding correctness under different perturbation strengths. For both single image super resolution and semantic segmentation, the BER remains 0.0000 across all evaluated settings of Noise, Jitter, JPEG recompression, and Gaussian blur. This indicates that the decoded bit sequence remains unchanged across perturbation levels, suggesting that the trigger-share decoding process is insensitive to these transmission induced distortions. Overall, the consistently zero BER across tasks and perturbation types demonstrates stable and deterministic decoding performance even when the input undergoes noticeable quality degradation.

The model trained with trigger injection is termed the triggered model. On the Carvana semantic segmentation task, the triggered model maintains segmentation utility on clean inputs. The clean baseline achieves a mean IoU of 0.9752 with a standard deviation of 0.0108 and a range from 0.9359 to 0.9923, and a mean Dice of 0.9874 with a standard deviation of 0.0056 and a range from 0.9669 to 0.9962. Evaluated on the same clean test set, the triggered model attains a mean IoU of 0.9750 with a standard deviation of 0.0109 and

a range from 0.9347 to 0.9939, and a mean Dice of 0.9873 with a standard deviation of 0.0056 and a range from 0.9663 to 0.9969, showing only negligible differences from the baseline. Output consistency on clean inputs is further assessed by directly comparing the predicted masks produced by the two models. The prediction IoU and Dice average 0.9811 and 0.9904, with standard deviations of 0.0080 and 0.0041 and ranges of 0.9483 to 0.9959 and 0.9735 to 0.9980, respectively. The pixel level XOR disagreement rate is 0.0041 on average with a standard deviation of 0.0022 and a range from 0.0010 to 0.0176, meaning that only about 0.41% of pixels differ between the two predictions on average. Fig. 10 visualizes these differences using XOR maps. Overall, these results suggest that, when the trigger is absent, trigger injection does not noticeably change normal segmentation behavior while preserving strong task performance.

To evaluate whether trigger injection alters normal inference behavior in the single image super resolution task, we feed the same clean test set into both the clean model and the triggered model and measure similarity between their outputs. With clean inputs, the triggered model remains highly consistent with the clean model, achieving an average PSNR of 33.56 dB, an SSIM of 0.9587, and an LPIPS of 0.0234. These results indicate that, in the absence of the trigger, trigger injection has a negligible impact on model utility.

Conclusion

We propose a model agnostic verification framework that operates in a black box setting and unifies model watermarking with access control. The key idea is to assign each user a distinct trigger and employ a trigger-to-share binding mechanism, so that the model outputs a decodable share pattern upon query. Verification can be performed solely through input–output queries by decoding the predicted share and matching it against the registered information, without requiring access to model weights, training data, or internal representations, making the framework well aligned with practical deployment scenarios such as API based or packaged models.

Experimental results show that the proposed method can reliably reconstruct the corresponding share patterns and achieves a zero bit error rate ($BER = 0$) on two representative tasks, single image super resolution and semantic segmentation. When the trigger is absent, the model’s normal inference behavior and task performance remain unchanged or differ only negligibly. Moreover, under common input perturbations, the decoded shares remain consistent, demonstrating the practicality and robustness of the framework. Future work will further investigate robustness against adaptive attackers and model changes after deployment (e.g., pruning, quantization, and fine tuning), and evaluate scalability across architectures and tasks.



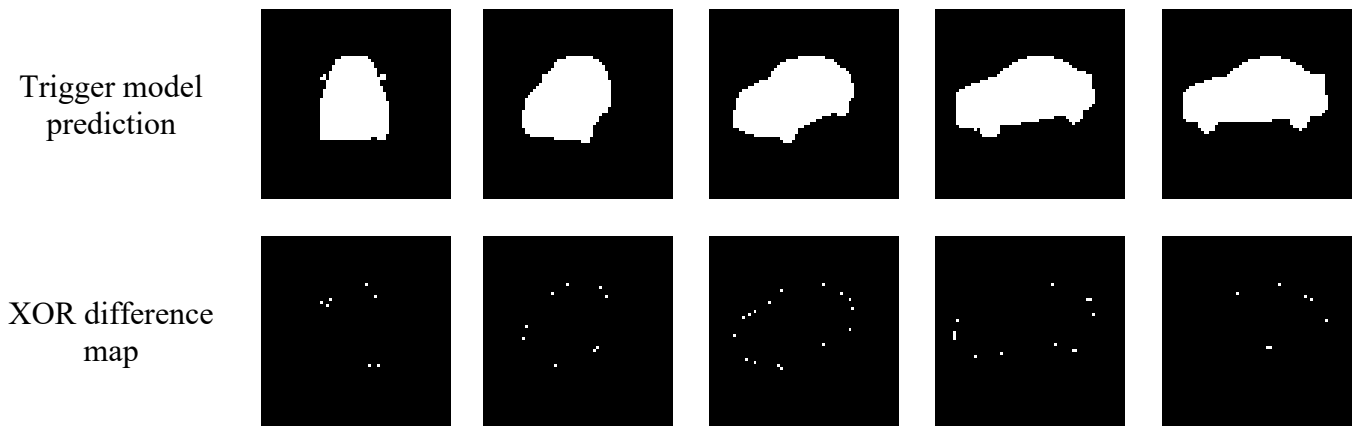


Fig. 10. Compares clean and trigger model outputs with XOR map.

Author Contributions

Conceptualization, K.G. and C.-C.C. (Ching-Chun Chang); methodology, S.-H.G., F.-Y.C., K.G. and C.-C.C. (Ching-Chun Chang); software, S.-H.G. and F.-Y.C.; validation, S.-H.G. and F.-Y.C.; writing—original draft preparation, S.-H.G. and F.-Y.C.; writing—review and editing, S.-H.G., F.-Y.C., K.G., C.-C.C. (Ching-Chun Chang) and C.-C.C. (Chin-Chen Chang). All authors have read and agreed to the published version of the manuscript.

Funding

No funding was received to assist with the preparation of this manuscript.

Conflict of Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data are contained within the article.

References

- [1] Li, Y., Wang, H. and Barni, M., 2021. A survey of deep neural network watermarking techniques. *Neurocomputing*, 461, pp.171-193.
- [2] Li, B., Fan, L., Gu, H., Li, J. and Yang, Q., 2022. FedIPR: Ownership verification for federated deep neural network models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(4), pp.4521-4536.
- [3] Wang, L. and Cheng, M., 2024, November. GuardEmb: Dynamic Watermark for Safeguarding Large Language Model Embedding Service Against Model Stealing Attack. In *Findings of the Association for Computational Linguistics: EMNLP 2024* (pp. 7518-7534).
- [4] Hu, A., Lu, Z., Xie, R. and Xue, M., 2023. VeriDIP: Verifying Ownership of Deep Neural Networks Through Privacy Leakage Fingerprints. *IEEE Transactions on Dependable and Secure Computing*, 21(4), pp.2568-2584.

- [5] Chang, C.C., Horng, J.H., Shih, C.S. and Chang, C.C., 2020. A maze matrix-based secret image sharing scheme with cheater detection. *Sensors*, 20(13), p.3802.
- [6] Li, L., Bai, R., Lu, J., Zhang, S. and Chang, C.C., 2021. A watermarking scheme for color image using quaternion discrete fourier transform and tensor decomposition. *Applied Sciences*, 11(11), p.5006.
- [7] Lin, C.C., Chang, C.C. and Wang, Z.M., 2019. Reversible data hiding scheme using adaptive block truncation coding based on an edge-based quantization approach. *Symmetry*, 11(6), p.765.
- [8] Fkirin, A., Attiya, G., El-Sayed, A. and Shouman, M.A., 2022. Copyright protection of deep neural network models using digital watermarking: a comparative study. *Multimedia Tools and Applications*, 81(11), pp.15961-15975.
- [9] Li, Y., Maharana, S.K. and Guo, Y., 2024, September. Not just change the labels, learn the features: watermarking deep neural networks with multi-view data. In *European Conference on Computer Vision* (pp. 273-289). Cham: Springer Nature Switzerland.
- [10] Xu, T., Wang, C., Liu, G., Yang, Y., Peng, K. and Liu, W., 2024. United We Stand, Divided We Fall: Fingerprinting Deep Neural Networks via Adversarial Trajectories. *Advances in Neural Information Processing Systems*, 37, pp.69299-69328.
- [11] Li, L., Wang, S., Zhang, S., Luo, T. and Chang, C.C., 2020. Homomorphic encryption-based robust reversible watermarking for 3D model. *Symmetry*, 12(3), p.347.
- [12] Adi, Y., Baum, C., Cisse, M., Pinkas, B. and Keshet, J., 2018. Turning your weakness into a strength: Watermarking deep neural networks by backdooring. In *27th USENIX security symposium (USENIX Security 18)* (pp. 1615-1631).
- [13] Zhang, J., Gu, Z., Jang, J., Wu, H., Stoecklin, M.P., Huang, H. and Molloy, I., 2018, May. Protecting intellectual property of deep neural networks with watermarking. In *Proceedings of the 2018 on Asia conference on computer and communications security* (pp. 159-172).
- [14] Le Merrer, E., Perez, P. and Trédan, G., 2020. Adversarial frontier stitching for remote neural network watermarking. *Neural Computing and Applications*, 32(13), pp.9233-9244.
- [15] Ngo, A.T., Heng, C.S., Chattopadhyay, N. and Chattopadhyay, A., 2025. Persistence of backdoor-based watermarks for neural networks: A comprehensive evaluation. *IEEE Transactions on Neural Networks and Learning Systems*.
- [16] Wei, C., Wang, Y., Gao, K., Shao, S., Li, Y., Wang, Z. and Qin, Z., 2024. Pointncbw: Towards dataset ownership verification for point clouds via negative clean-label backdoor watermark. *IEEE Transactions on Information Forensics and Security*.
- [17] Pegoraro, A., Segna, C., Kumari, K. and Sadeghi, A.R., 2024. {DeepEclipse}: How to Break {White-Box} {DNN-Watermarking} Schemes. In *33rd USENIX Security Symposium (USENIX Security 24)* (pp. 5287-5304).
- [18] Puah, Y.H., Ngo, A.T., Chattopadhyay, N. and Chattopadhyay, A., 2024. BlockDoor: Blocking Backdoor Based Watermarks in Deep Neural Networks. *arXiv preprint arXiv:2412.12194*.
- [19] Li, F., Zhao, H., Du, W. and Wang, S., 2024, March. Revisiting the information capacity of neural network watermarks: Upper bound estimation and beyond. In *Proceedings of the AAAI Conference on Artificial Intelligence* (Vol. 38, No. 19, pp. 21331-21339).

- [20] Gu, T., Liu, K., Dolan-Gavitt, B. and Garg, S., 2019. Badnets: Evaluating backdooring attacks on deep neural networks. *IEEE Access*, 7, pp.47230-47244.
- [21] Li, Y., Jiang, Y., Li, Z. and Xia, S.T., 2022. Backdoor learning: A survey. *IEEE transactions on neural networks and learning systems*, 35(1), pp.5-22.
- [22] Li, Y., Li, Y., Lv, Y., Jiang, Y. and Xia, S.T., 2021. Hidden backdoor attack against semantic segmentation models. *arXiv preprint arXiv:2103.04038*.
- [23] Ding, Y., Wang, Z., Qin, Z., Zhou, E., Zhu, G., Qin, Z. and Choo, K.K.R., 2023. Backdoor attack on deep learning-based medical image encryption and decryption network. *IEEE Transactions on Information Forensics and Security*, 19, pp.280-292.
- [24] Chan, S.H., Dong, Y., Zhu, J., Zhang, X. and Zhou, J., 2022, October. Baddet: Backdoor attacks on object detection. In *European conference on computer vision* (pp. 396-412). Cham: Springer Nature Switzerland.
- [25] Nguyen, T.T., Quoc Viet Hung, N., Nguyen, T.T., Huynh, T.T., Nguyen, T.T., Weidlich, M. and Yin, H., 2024. Manipulating recommender systems: A survey of poisoning attacks and countermeasures. *ACM Computing Surveys*, 57(1), pp.1-39.
- [26] Shamir, A., 1979. How to share a secret. *Communications of the ACM*, 22(11), pp.612-613.
- [27] Ning, J., Huang, X., Susilo, W., Liang, K., Liu, X. and Zhang, Y., 2020. Dual access control for cloud-based data storage and sharing. *IEEE Transactions on Dependable and Secure Computing*, 19(2), pp.1036-1048.
- [28] Yang, C.N. and Lai, J.B., 2013, July. Protecting data privacy and security for cloud computing based on secret sharing. In *2013 International Symposium on Biometrics and Security Technologies* (pp. 259-266). IEEE.
- [29] Singh, P., Agarwal, N. and Raman, B., 2018. Secure data deduplication using secret sharing schemes over cloud. *Future Generation Computer Systems*, 88, pp.156-167.
- [30] Iftene, S., 2007. General secret sharing based on the chinese remainder theorem with applications in e-voting. *Electronic Notes in Theoretical Computer Science*, 186, pp.67-84.
- [31] Liu, Y. and Zhao, Q., 2019. E-voting scheme using secret sharing and K-anonymity. *World Wide Web*, 22(4), pp.1657-1667.
- [32] Raman, R.K. and Varshney, L.R., 2018, February. Distributed storage meets secret sharing on the blockchain. In *2018 information theory and applications workshop (ITA)* (pp. 1-6). IEEE.
- [33] Wang, H., Zhang, Y., Cheng, Y., Li, Q., Zhao, J. and Li, W., 2023, July. A data privacy protection scheme integrating federal learning and secret sharing. In *2023 IEEE 5th International Conference on Power, Intelligent Computing and Systems (ICPICS)* (pp. 311-315). IEEE.
- [34] Zhu, H., Goh, R.S.M. and Ng, W.K., 2020. Privacy-preserving weighted federated learning within the secret sharing framework. *IEEE Access*, 8, pp.198275-198284.
- [35] Le, P.T., Pham, B.T., Chang, C.C., Hsu, Y.C., Tai, T.C., Li, Y.H. and Wang, J.C., 2023. Anti-aliasing attention U-net model for skin lesion segmentation. *Diagnostics*, 13(8), p.1460.
- [36] @kaggle. (2026). *Carvana Image Masking Challenge*. [online] Available at: <https://www.kaggle.com/c/carvana-image-masking-challenge>.

[37] Cocodataset.org. (2020). *COCO - Common Objects in Context*. [online] Available at: <https://cocodataset.org/#home>. [Accessed 14 Jan. 2026].